

COGNOME:

NOME:

MATRICOLA:

Intermedio di Algoritmi e Strutture Dati (9CFU), 4 Aprile 2024.

Esercizio 1 [5 punti]Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

- (A) $\sqrt{n}, n \log n, n^{\log n}, 2^{\log n^2}, 2^n$
- (B) $n^{4/3}, \binom{n}{2}, n(\log n)^3, 2^{\sqrt{\log n}}, n^{\log n}$
- (C) $n^{4/3}, n(\log n)^3, \binom{n}{2}, 2^{\sqrt{\log n}}, n^{\log n}$
- ~~(D)~~ $\sqrt{n}, \log n!, n^2 \log n, \binom{n}{n-3}, 2^{\sqrt{n}}$

Esercizio 2 [5 punti]

Si consideri l'algoritmo USELESS descritto di seguito, il cui input è un intero $n \geq 0$ potenza di 4. Sia $T(n)$ il tempo di esecuzione nel caso peggiore di USELESS con input n . Assumendo che il costo di sommare due interi sia $O(1)$, indipendentemente dagli interi considerati, quale delle seguenti affermazioni è vera?

Algorithm USELESS(n)

```

1: if  $n \leq 1$  then  $\rightarrow \Theta(1)$ 
2:   return  $n$ 
3: end if
4:  $sum = 0$ 
5: for  $i = 0$  to  $n^3 - 1$  do  $\rightarrow \Theta(n^3 - 1) = \Theta(n^3)$ 
6:    $sum = sum + 1$ 
7: end for
8: return USELESS( $3n/4$ ) + USELESS( $3n/4$ ) +  $sum$ 
```

$$T(n) = 2T(3n/4) + \Theta(n^3)$$

$$a=2 \quad b=4/3 \quad n^{\log_{4/3} 2} = n^{2.04} < n^3$$

$$T(n) = \Theta(n^3)$$

- ~~(A)~~ $T(n) = \Theta(n^3)$
- (B) $T(n) = \Theta(n^{\log_{4/3} 2})$
- (C) Nessuna delle altre
- (D) $T(n) = \Theta(n^{\log_{3/4} 2})$

Esercizio 3 [5 punti]

Indicare tutte e sole le affermazioni vere.

- (A) È possibile ordinare una sequenza arbitraria di 6 interi tramite al più 9 confronti.
- ~~(B)~~ Siano $A[1..n]$ e $B[1..n]$ due array, entrambi di n interi, tali che ciascuno dei loro elementi è compreso tra 1 e n . Sia $C[1..n]$ un array tale che $C[i] = \underbrace{A[i] \cdot B[i]}_{1 \dots n^2}$ per ogni $i = 1, \dots, n$. È possibile ordinare l'array C in $O(n)$. $\rightarrow$ Radix Sort
- (C) Sia $A[1..n]$ un array di n interi distinti tali che $A[1] < A[2] < \dots < A[n]$ e $A[1] > 0$. Un qualsiasi algoritmo che trovi un indice i tale che $A[i] = i$ oppure indichi che tale indice non esiste richiede $\Omega(\log n)$ confronti nel caso peggiore.
- ~~(D)~~ BINARY-SEARCH è un algoritmo ottimo.

Esercizio 4 [5 punti]

Indicare quale delle seguenti affermazioni è vera.

- ~~(A)~~ È possibile implementare una struttura dati del tipo LIFO tramite un min-heap.
- (B) Possiamo utilizzare un heap per ordinare in tempo lineare un insieme di numeri.
- (C) La procedura di max-heapify può essere lanciata su qualsiasi albero binario quasi completo.

(D) Utilizziamo sempre un max-heap per implementare una coda di priorità.

Esercizio 5 [5 punti]

Un albero binario è:

- ~~(A)~~ Un albero tale che ogni nodo ha al più due figli.
- (B) Un albero per la memorizzazione e ricerca di numeri binari.
- (C) Un albero tale che, dato un nodo x , il figlio sinistro ha un valore minore del valore associato al nodo x e il figlio destro ha un valore maggiore del valore associato al nodo x .
- (D) Un albero tale che ogni nodo è associato ad un valore maggiore di tutti i suoi figli.

SBARRAMENTO - 15 punti sugli esercizi precedenti

Esercizio 6 [10 punti]

Un array $A[1..n]$ di n interi distinti è *quasi ordinato* se ciascuno dei suoi elementi è fuori posto rispetto all'ordinamento crescente di al più una posizione. In altre parole, un qualsiasi elemento che nell'array ordinato occuperebbe la posizione i , in un array quasi ordinato può occupare una qualsiasi delle posizioni $i - 1, i, i + 1$. Per esempio, l'array $\langle 4, 1, 7, 10, 8 \rangle$ è quasi ordinato, mentre l'array $\langle 1, 7, 8, 4, 10 \rangle$ non lo è.

Vogliamo sviluppare un algoritmo efficiente che, dati in input un array $A[1..n]$ quasi ordinato ed un intero k , restituisca l'indice di k in A , se esiste, o il valore -1 , se k non appare in A .

1. Spiegare brevemente perchè un algoritmo che risolve il nostro problema in $\Theta(\log n)$ è ottimo. [4 punti]
2. Siano $A[1..n]$ un array quasi ordinato di n interi distinti, k un intero e m un indice tale che $1 < m < n$. Dimostrare che, se k appare in A e $A[m] > k$, allora: o $A[m - 1] = k$, o $A[m + 1] = k$, oppure k appare in $A[1..m - 2]$. [3 punti]
3. Usando l'osservazione del punto 2, scrivere lo pseudo-codice di un algoritmo Divide et Impera che risolve il nostro problema in $\Theta(\log n)$. [3 punti]

Esercizio 1 [5 punti]

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

(A) $\sqrt{n}$, $n \log n$, $n^{\log n}$, $2^{\log n^2}$, 2^n

(B) $n^{4/3}$, $\binom{n}{2}$, $n(\log n)^3$, $2^{\sqrt{\log n}}$, $n^{\log n}$

(C) $n^{4/3}$, $n(\log n)^3$, $\binom{n}{2}$, $2^{\sqrt{\log n}}$, $n^{\log n}$

(D) $\sqrt{n}$, $\log n!$, $n^2 \log n$, $\binom{n}{n-3}$, $2^{\sqrt{n}}$

A. $\sqrt{n} \leq n \log n \leq n^{\log n} > n^2$

• $\sqrt{n} = n^{1/2}$

• $2^{\log n^2} = 2^{2 \log n} = n^2$

B. $n^{4/3} \leq n^2 > n(\log n)^3$

• $\binom{n}{2} = n^2$

• $n(\log n)^3$

C. $n^{4/3} > n(\log n)^3$

D. $\sqrt{n} \leq \log n! \leq n^2 \log n \leq n^3 \leq 2^{\sqrt{n}}$

• $\sqrt{n} = n^{1/2}$

• $\log n! = n \log n$

• $\binom{n}{n-3} = \binom{n}{3} = n^3$

• $2^{\sqrt{n}}$

applico il log.: $\log n^3$ $\log 2^{\sqrt{n}}$
 $3 \log n$ $\sqrt{n} \log 2$
 $\log n < \sqrt{n}$

Esercizio 6 [10 punti]

Un array $A[1..n]$ di n interi distinti è *quasi ordinato* se ciascuno dei suoi elementi è fuori posto rispetto all'ordinamento crescente di al più una posizione. In altre parole, un qualsiasi elemento che nell'array ordinato occuperebbe la posizione i , in un array quasi ordinato può occupare una qualsiasi delle posizioni $i-1, i, i+1$. Per esempio, l'array $\langle 4, 1, 7, 10, 8 \rangle$ è quasi ordinato, mentre l'array $\langle 1, 7, 8, 4, 10 \rangle$ non lo è.

Vogliamo sviluppare un algoritmo efficiente che, dati in input un array $A[1..n]$ quasi ordinato ed un intero k , restituisca l'indice di k in A , se esiste, o il valore -1 , se k non appare in A .

1. Spiegare brevemente perchè un algoritmo che risolve il nostro problema in $\Theta(\log n)$ è ottimo. [4 punti]
2. Siano $A[1..n]$ un array quasi ordinato di n interi distinti, k un intero e m un indice tale che $1 < m < n$. Dimostrare che, se k appare in A e $A[m] > k$, allora: o $A[m-1] = k$, o $A[m+1] = k$, oppure k appare in $A[1..m-2]$. [3 punti]
3. Usando l'osservazione del punto 2, scrivere lo pseudo-codice di un algoritmo Divide et Impera che risolve il nostro problema in $\Theta(\log n)$. [3 punti]

1. Sappiamo che l'algoritmo Binary-Search è ottimo poiché impiega un numero di Confronti pari a $O(\log n)$ nel caso peggiore per stabilire se la chiave k è contenuta o meno nell'array ordinato.

2. Dato che l'array è quasi ordinato, l'elemento k che nell'array ordinato occuperebbe la posizione i può trovarsi solo in $A[i-1]$ o $A[i]$ o $A[i+1]$.
se troviamo $A[m] > k$, ci sono 3 possibilità:

1. k è in $A[m+1]$

2. k è in $A[m-1]$

3. k si trova necessariamente nel tratto precedente $A[1..m-2]$, poiché tutti gli elementi da $A[m+1]$ in poi sono necessariamente maggiori di $A[m+1]$ e quindi anche di k .

3. Trova(k, A, p, q)

if $p > q$ return -1

$m = \lfloor \frac{p+q}{2} \rfloor$

if $A[m] == k$ return m

if $m+1 \leq q$ and $A[m+1] == k$ return $m+1$

if $m-1 \geq p$ and $A[m-1] == k$ return $m-1$

if $A[m] > k$ return Trova($k, A, p, m-2$)

else return Trova($k, A, m+2, q$)